Task1:

Code:

```
% Load the T_junction_best.mat file and extract S-parameter and frequency data
data = load('T_junction_best.mat');
freq = data.freq;    % Frequency
S11_val = data.S11db;    % S11 in dB
S21_val = data.S21dB;    % S21 in dB
S31_val = data.S31db;    % S31 in dB

% Define the evaluation function
function perf = evaluate(f, S11_val, S21_val, S31_val)
    % Define frequency ranges and design specifications
    S11_spec1 = -20;    % S11 threshold for 12.5~12.75 GHz and 13~13.25 GHz
    S21_spec1 = -20;    % S21 threshold for 12~12.3 GHz
    S21_spec2 = -55;    % S21 threshold for 13~13.25 GHz
    S31_spec1 = -55;    % S31 threshold for 12.5~12.75 GHz
    S31_spec2 = -20;    % S31 threshold for 13.4~13.75 GHz

    % Initialize performance evaluation result
    perf = zeros(1, 6);

    % Calculate deviations for S11
    E1 = max(S11_val(f >= 12.5 & f <= 12.75) - S11_spec1);
    E2 = max(S11_val(f >= 13 & f <= 13.25) - S11_spec1);
    if E1 <= 0, E1 = 0; end
    if E2 <= 0, E2 = 0; end
    perf(1) = E1;    % Record deviation value
    perf(2) = E2;

    % Calculate deviations for S21
    E3 = max(S21_val(f >= 12 & f <= 12.3) - S21_spec1);
    E4 = max(S21_val(f >= 13 & f <= 13.25) - S21_spec2);
    if E3 <= 0, E3 = 0; end
    if E4 <= 0, E4 = 0; end
    perf(3) = E3;
    perf(4) = E4;

    % Calculate deviations for S31
    E5 = max(S31_val(f >= 12.5 & f <= 12.75) - S31_spec1);
    E6 = max(S31_val(f >= 13.4 & f <= 13.75) - S31_spec2);
    if E5 <= 0, E5 = 0; end
    if E6 <= 0, E6 = 0; end
    perf(5) = E5;
    perf(6) = E6;
```

```
end

% Run the evaluation function
perf = evaluate(freq, S11_val, S21_val, S31_val);
disp('Performance evaluation result:');
disp(perf);
```

Output:

```
>> task1
Performance evaluation result:
    4.0451    1.5605         0         0         0    1.7390
```

Explanation

In task 1, I developed an evaluation function to assess the performance of a microwave diplexer based on specific S-parameter requirements (S11, S21, and S31). This function will help determine if the diplexer meets the given design specifications and measure any deviations

The performance output [4.0451, 1.5605, 0, 0, 0, 1.7390] shows deviations within each frequency range. Here, E1 (4.0451) and E2 (1.5605) indicate S11 deviations, while E6 (1.7390) indicates an S31 deviation, signifying instances where values exceed specified thresholds. Values of 0 in E3, E4, and E5 confirm compliance with the requirements in those ranges.

Task2

Code:

```
%============================================================
================
% This function is based on Julien Le Kernec's Spectrogram feature extraction
% and batch processing code Rev 2022.
% authors Zhenghui Li/Julien Le Kernec - July 2022 - University of Glasgow
%============================================================
================

function [Spec_image,labels] =
spectrogramprops2(Clipping,Hamming,TW,OF,Te,Diff,Comb1,Comb2,Comb3,Comb4,Comb
5,Comb6)

%This is a combination function in order to
%a)extract, select and fuse features from radar data.
%b)adjust the parameters and give a 10-fold cross validation error rate to
%evaluate the parameters.

%Before running the program, make sure that 'natsortfile.m' has been added
%to the MATLAB path or is in the same folder.
```

```
%The FOLLOWING PARAMETERS have been added
%
%   'Clipping'              This is the clipping factor of the spectrogram,
%                             The spectrogram would be clipped according to its time axis.
%                             This factor is between 0 (No clippling), 1.5(clipping with 1.5s),
%                             3(Clipping with 3s) and 5(Clipping with 5s).
%   'Hamming'                This factor decides whether the hamming window will
%                             be used or not. Hamming == 1 means using hamming window,
%                             other input number means Hamming window will not be used.
%   'TW'                     Time window lenght factor decides the time window length
%                             in STFT of the spectrogram. There are five options: 150
%                             300,450,600,750.Range:[100 1000]
%   'OF'                     The overlap factor is also used in STFT process.
%                             The default value is 0.95 and it is
%                             adjustable.Range[0.5 0.95]
%   'Te'                     An interger between -10 to 20.This is the thresholding factor.
%                             The adaptive thresholding method can calculate a threshold T
%                             This T can be changed (add or minus) to improve
%                             the accuracy of classification.[T-20 T+20]
%   'Diff'                  This factor is used in adaptive thresholding method calculation
%                             to determine the thresholding value. The default input
%                             value should be 0.1. [0.01 1]
%   'Comb1-6'                Comb 1 to Comb 6 are correspoding to features of mask,
%                             masked phase, maske unwrapped phase, masked spectrogram
%                             image,spectrogram and masked spetrogram, these six
%                             different types of features. Input 1 means correspoding
%                             types of feature will be used, Input 0 means it will
%                             not be used..

% There are 443 variables in this function in total.
Spec_image = spectrogramprops(0,1,256,0.95,0,0.1,0,0,0,0,1,0);
% This means human data is chosen and the spectrogram has no clippinig. The
% Hamming window is used and time windwo length is 150ms, with 0.95 overlap
% factor.The final thresholding value is the adaptive thresholding method
% result with 0.1 difference. 6 different types of features are used (Comb1
% to Comb6).

%% Extract label from files
%============================================================
================

% input description of format:
% the input can be a string or a cell. when a cell is received, the
% function will extract the label from every element of the cell.
```

```
% the filename xPxxAxxRxx.xxx tells this file records which activity, performed by which
% person and how many repetition time it is.

%output number 1 values and meaning: one-dimension array. each element has
% the value from 1 to 6 such as 1-walking, 2-sitting, 3-standing, 4-drink water,
% 5-pick, 6-fall.

%
%================================================================
================

Load_path = strcat(pwd,'\1 December 2017 Dataset\'); % path where saved radar(.dat) file
%% Ancortek reading part
% [filename,pathname] = uigetfile('*.dat');
Files = dir(strcat(Load_path,'*.dat'));
Files = natsortfiles(Files); % sorts files in alphanumeric order
LengthFiles = length(Files);
%% Extract label from files
%================================================================
================

% input description of format:
% the input can be a string or a cell. when a cell is received, the
% function will extract the label from every element of the cell.

% the filename xPxxAxxRxx.xxx tells this file records which activity, performed by which
% person and how many repetition time it is.

%output number 1 values and meaning: one-dimension array. each element has
% the value from 1 to 6 such as 1-walking, 2-sitting, 3-standing, 4-drink water,
% 5-pick, 6-fall.

%
%================================================================
================

labels = zeros(LengthFiles, 1); % to ensure that Matlab runs faster, it is good practice to pre-
allocate memory to a vector or matrix.
% The labels variable will be a column vector of the same size as the number of radar
signatures given by the variable LenghFiles
% create a column vector filled with zeros to record the labels as you go through the database
%%
for i = 1:LengthFiles
```

```
fileID = fopen(strcat(Load_path,Files(i).name));
%% from the description above, you kow that the label is the first character of the name of the file
% however it is a character not a number
% read only the first character of the string and use the function str2num
% to convert it into a number to be saved at the i_th row of the labels vector
labels(i) = str2num(Files(i).name(1));
%% the raw data files first 4 rows contain the radar parameters used for the recording
% we first scan the file to read the information from start to finish and
% save it in dataArray which is a cell you first need to transfer the cell
% data into a column vector radar data
dataArray = textscan(fileID, '%f');
fclose(fileID);
radarData = dataArray{1};
% fileID = [];
% dataArray = [];
fc = radarData(1); % the first row contains the center frequency in Hz - you need to pass the value of the first row to fc
Tsweep = radarData(2); % the second row contains the Sweep time in ms - you need to pass the value of the 2nd row to Tsweep
Tsweep= radarData(2) / 1000; %You will need Tsweep in seconds for the rest of the program make sure you do the conversion from ms to s.
NTS = radarData(3); % the third row contains the Number of time samples per sweep - you need to pass the value of the third row to NTS
Bw = radarData(4); % the 4th row is the FMCW Bandwidth - you need to pass the value of the 4th row to Bw
Data = radarData(5:end); % finally the data from the 5th row to the end of the file is raw data in I+j*Q complex format that you need to pass to Data
% fs=NTS/Tsweep; % sampling frequency ADC
record_length=length(Data)/NTS*Tsweep; % length of recording in s
nc=record_length/Tsweep; % number of chirps

%% Reshape data into chirps and do range FFT (1st FFT)
% The first FFT needs to be performed across all the sweeps
% you could create a for loop but this is very inefficient in Matlab
% instead we are going to use the matrix calculations from matlab
% to achieve this you need to first reshaped the vector into a matrix
% Data_timebis of dimensions NTS x nc ie sweep length by the number of
% sweeps
Data_timebis = reshape(Data, NTS, []);
% Moving target calculation
Data_time = Data_timebis(:,2:end)-Data_timebis(:,1:end-1);

%Hamming window prior to FFT may reduce the sidelobes in range
```

```
if Hamming==1
    win = repmat(hamming(NTS),1,size(Data_time,2));
else
    win = ones(NTS,size(Data_time,2));
end

%Part taken from Ancortek code for FFT and IIR filtering
%% You need to apply the window function to the raw data in order to smooth the FFT result
- you need to perform an elementwise multiplication between Data_time and win
Data_time = Data_time .* win;
%% you then need to perform an FFT of size NTS in the first dimension of the matrix Data over
the entire matrix
tmp = fft(Data_time, NTS, 1);
%% from here on do not modify anything until you reach the feature extraction stage within
Data_range_MTI= tmp(1:NTS/2,:);
% Data_range_MTI = Data_range(:,2:end)-Data_range(:,1:end-1);
% Data_range_MTI = gpuArray(Data_range_MTI);

% Draw the graph of range-time figure
% figure(1)
% colormap(jet)
% imagesc(slow_time,range_axis,20*log10(abs(Data_range_MTI)))
% xlabel('Time[s]', 'FontSize',16);
% ylabel('Range [m]','FontSize',16)
% title('Range Profiles after MTI filter')
% clim = get(gca,'CLim'); axis xy; ylim([1 size(Data_range_MTI,1)])
% axis([min(slow_time) max(slow_time) min(range_axis) max(range_axis)])
% set(gca, 'CLim', clim(2)+[-40,+Inf]);
% colorbar
% drawnow
% saveas(gcf,[Range_time_Image_path,Files(i).name,'.png']);
% delete(gcf);

%% Spectrogram processing for 2nd FFT to get Doppler
% This selects the range bins where we want to calculate the spectrogram
bin_indl = 10;
bin_indu = 60;
%Parameters for spectrograms
MD_PRF=1/Tsweep;
MD_TimeWindowLength = TW;
MD_OverlapFactor = OF;
MD_OverlapLength = round(MD_TimeWindowLength*MD_OverlapFactor);
PF = 4;
```

```
MD_Pad_Factor = PF;
MD_FFTPoints = MD_Pad_Factor*MD_TimeWindowLength;
MD_DopplerBin=MD_PRF/(MD_FFTPoints);
MD_DopplerAxis=-MD_PRF/2:MD_DopplerBin:MD_PRF/2-MD_DopplerBin;
MD_WholeDuration=size(Data_range_MTI,2)/MD_PRF;
%MD_NumSegments=floor((size(Data_range_MTI,2)-
MD_TimeWindowLength)/floor(MD_TimeWindowLength*(1-MD_OverlapFactor)));

%Method 1 - COHERENT SUM
myvec_MTI=sum(Data_range_MTI(bin_indl:bin_indu,:));
% myvec=sum(Data_range(bin_indl:bin_indu,:));
Data_spec_MTI2=fftshift(spectrogram(myvec_MTI,MD_TimeWindowLength,MD_OverlapLen
gth,MD_FFTPoints),1);
%
Data_spec_P=fftshift(spectrogram(myvec,MD_TimeWindowLength,MD_OverlapLength,MD_F
FTPoints),1);

%Method 2 - SUM OF RANGE BINS
% Data_spec_MTI2_J=0;
% Data_spec2_J=0;
% for RBin=bin_indl:1:bin_indu
%                                                    Data_MTI_temp_J          =
fftshift(spectrogram(Data_range_MTI(RBin,:),MD_TimeWindowLength,MD_OverlapLength,M
D_FFTPoints),1);
%       Data_spec_MTI2_J=Data_spec_MTI2_J+Data_MTI_temp_J;
%                                                    Data_temp                =
fftshift(spectrogram(Data_range(RBin,:),MD_TimeWindowLength,MD_OverlapLength,MD_FFT
Points),1);
%     Data_spec2=Data_spec2+Data_temp;
% end

MD_TimeAxis=linspace(0,MD_WholeDuration,size(Data_spec_MTI2,2));

% Normalise and plot micro-Doppler
Data_spec_MTI2 = flipud(Data_spec_MTI2);
Data_spec_MTI2 = Data_spec_MTI2./max(max(Data_spec_MTI2));

%Hightlight the velocity range of target
Time_Lim = MD_DopplerAxis.*3e8/2/5.8e9;
Lim = find(abs(Time_Lim)<6);
max_Dop = max(Lim);
min_Dop = min(Lim);
Data_spec_MTI2 = Data_spec_MTI2(min_Dop:max_Dop,:);
MD_DopplerAxis = MD_DopplerAxis(:,min_Dop:max_Dop);
```

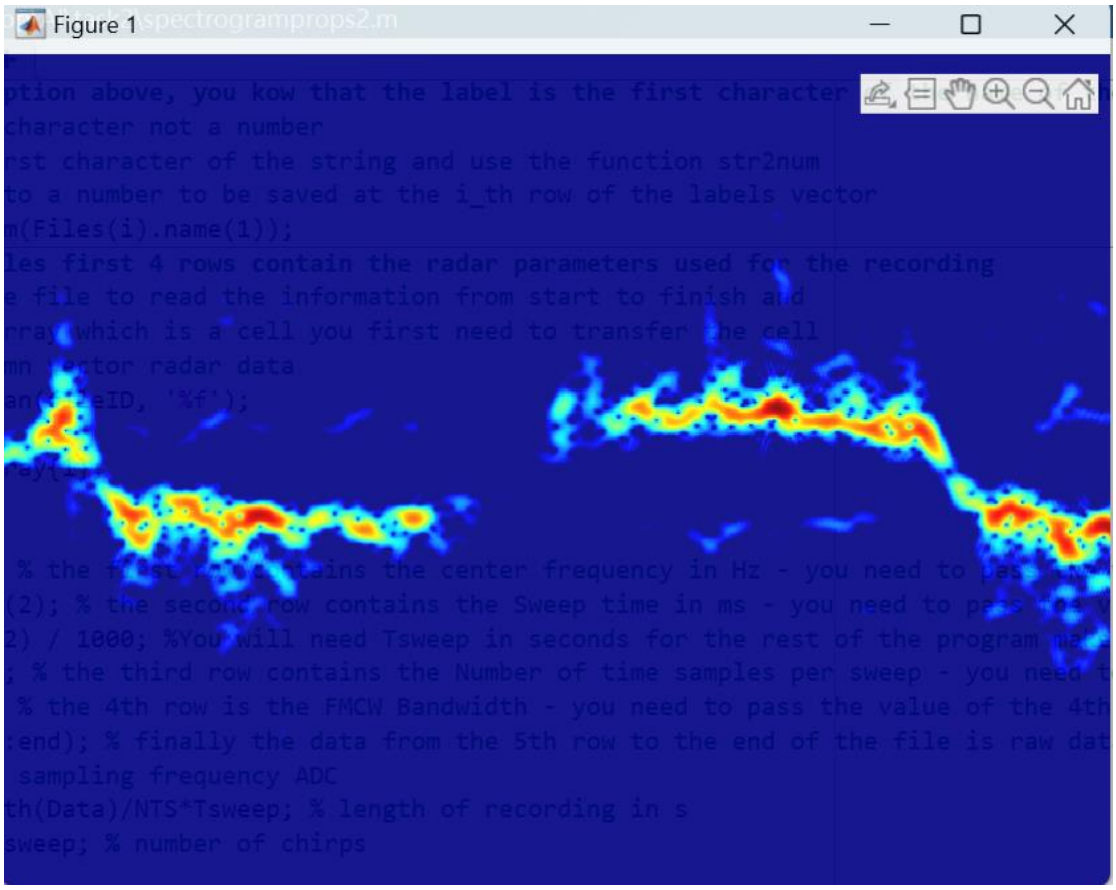
```
Time = max(MD_TimeAxis);
spec_Shape = size(Data_spec_MTI2);
each = floor(spec_Shape(2)/Time); %The number of columns corresponding to 1s.
p = floor(Time/Clipping);


if Clipping == 0
    feature{i} = nodivision(Data_spec_MTI2,MD_DopplerAxis,MD_TimeAxis,Te,Diff,Comb1,Comb2,Comb3,Comb4,Comb5,Comb6,OF,TW,PF,Files(i).name);
else
    feature{i} = division(p,Clipping,each,Data_spec_MTI2,MD_DopplerAxis,MD_TimeAxis,Te,Diff,Comb1,Comb2,Comb3,Comb4,Comb5,Comb6,OF,TW,PF,Files(i).name);
end

end

%% convert feature cell to matrix
% the features come out as a cell and need to be converted to a matrix, the
% features need to be concatenated vertically in a for loop to create the
% Spec_image matrix of size (number of samples x number of features)
cell_size = size(feature);
matrix_sample = feature{1}; % convert the first feature cell feature(1) into a matrix
sample_size = size(matrix_sample);
matrix_sample = zeros(1,sample_size(2));
for u = 1:cell_size(2)
    matrix_feature = feature{u};% convert the first feature cell feature(u) into a matrix
    matrix_sample = vertcat(matrix_sample,matrix_feature);
end
matrix_sample(1,:) = [];
Spec_image = vertcat(feature{:}); %copy the contatenated feature matrix into the output variable Spec_image
end
```

output



It generated many images. I just put one of them here.

Task3

Code:

```
% Define Ackley function parameters
a = 20;
b = 0.2;
c = 2 * pi;

% Define the Ackley function for 10-dimensional optimization
ackleyFunc = @(x) -a * exp(-b * sqrt(sum(x.^2) / 10)) - exp(sum(cos(c * x)) / 10) + a + exp(1);

% Set the number of dimensions
n = 10;

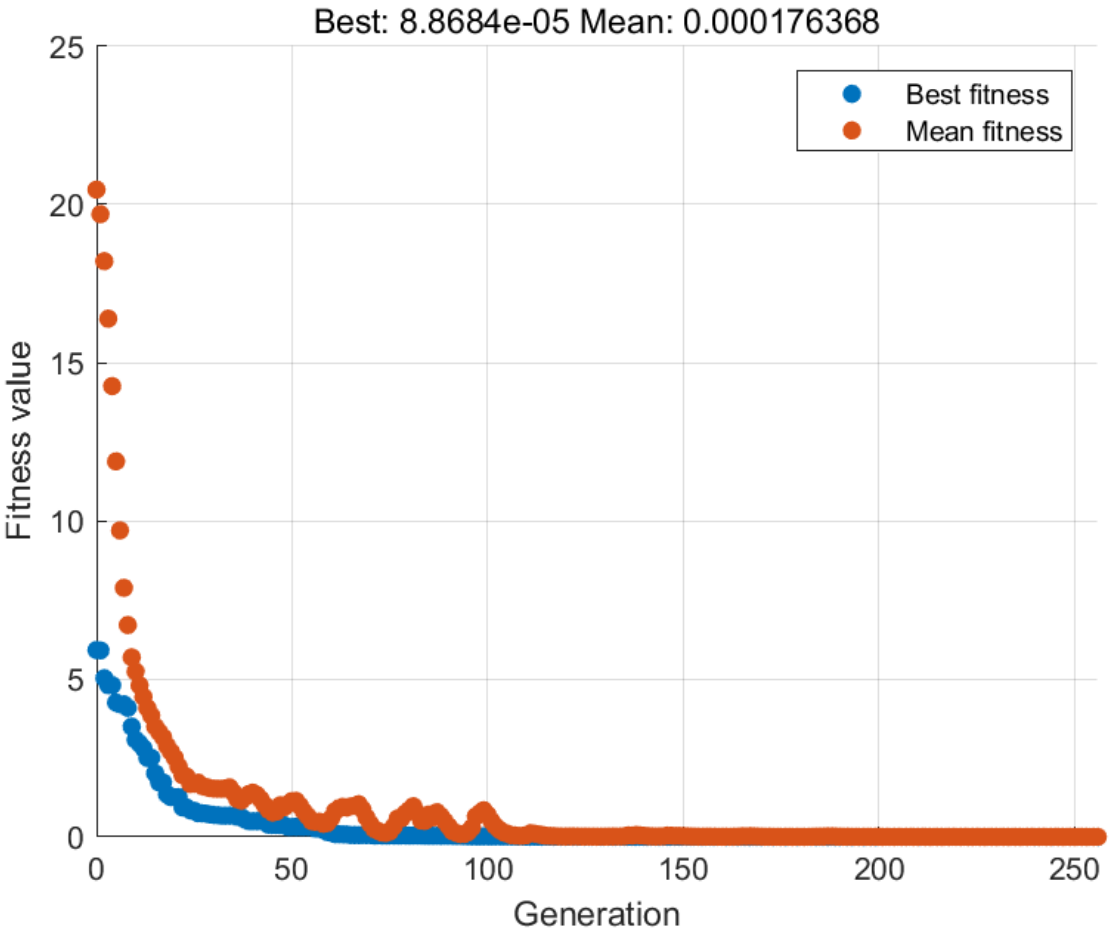
% Define search range for each dimension
lb = -15 * ones(1, n);    % Lower bound
ub = 30 * ones(1, n);     % Upper bound

% Configure options for the genetic algorithm
options = optimoptions('ga', 'Display', 'iter', 'PlotFcn', @gaplotbestf);
```

```
% Use genetic algorithm to find the minimum value of the Ackley function
[x_opt, fval] = ga(ackleyFunc, n, [], [], [], [], lb, ub, [], options);

% Display the optimized results
disp('Optimal decision variables (x):');
disp(x_opt);    % Display optimal values for decision variables
disp('Optimal objective function value:');
disp(fval);    % Display the minimum value achieved for Ackley function
```

output



Optimal decision variables (x):

1.0e-04 *=0.0001

0.1639 0.0081 0.4143 0.0004 0.0024 0.0339 0.5371 0.0170 0.0356 0.0388

Optimal objective function value:

8.8684e-05=0.000088684

Explanation

The goal of Task 3 is to use a genetic algorithm (GA) to optimize a 10-dimensional Ackley function, a standard benchmark in optimization. By finding the function's minimum value

within a specified range, GA simulates an evolutionary process to locate the best decision variable set that minimizes the Ackley function.